Real-Time California Grid Carbon Pipeline: Open, Campus-Scale Carbon-Aware Scheduling with Kafka, Spark Structured Streaming, Redis and Snowflake

Sayli Suryakant Bhongle, Karishma Routhu, Shashidhar Kumar Chintagunta, Vamsi Krishna Bethapudi
Department of Applied Data Science, San José State University, San José, CA, USA
{firstname.lastname}@sjsu.edu

***Abstract*—The carbon intensity of a kilowatt-hour in California changes several-fold within a day: solar pushes it low around noon, and gas plants raise it in the evening. Most campus loads (EV charging, research computing, scheduled lab equipment) ignore this. We present an open pipeline built only on public data that (i) ingests CAISO five-minute fuel-mix data, EIA-930 hourly data for every U.S. balancing authority, and NWS weather forecasts through Kafka; (ii) computes average carbon intensity with Spark Structured Streaming using event-time windows and watermarks that absorb CAISO's data revisions; (iii) serves a live signal and a ranked 24-hour outlook from Redis while keeping the full history in a Snowflake star schema built with dbt; and (iv) drives an Airflow sensor that delays flexible jobs into clean hours. Streaming algorithms (Bloom filter, DGIM, reservoir sampling), locality-sensitive hashing for analog-day forecasting, and k-anonymity with differential privacy for load data are integrated into the pipeline. [TODO: one sentence with the headline results: median latency, accuracy vs. CAISO, forecast MAE, kg CO_2 avoided].**

***Index Terms*—carbon-aware computing, stream processing, Spark Structured Streaming, Kafka, data warehouse, CAISO**

I. Introduction

California's grid regularly produces more solar at midday than it can use; CAISO curtailed more than 2.7 million MWh of wind and solar in the first five months of 2025 [1]. Evenings, by contrast, still rely heavily on natural gas. The same kWh can therefore be nearly carbon-free at 1 p.m. and much dirtier at 7 p.m. A campus that moves flexible load into clean hours reduces emissions without using less energy. The obstacle is timing and packaging, not data: the signals are spread over CAISO, EIA and weather services, arrive at different speeds and are rarely in a form a facilities manager or job scheduler can act on.

Contributions. (1) An end-to-end open pipeline from public feeds to a live carbon signal and a historical warehouse; (2) handling of CAISO revisions and time-zone/DST differences with watermarks, idempotent merges and UTC normalisation; (3) three explainable day-ahead forecasters, including an LSH analog-day method; (4) a carbon-aware Airflow sensor and a measured savings study; (5) privacy-preserving release of load data. Code: **[TODO: public GitHub URL]**.

II. Related Work

A. Carbon-aware computing

Radovanović et al. describe Google's use of grid carbon-intensity forecasts to shift flexible data-centre load [2]. Wiesner et al. show by simulation that postponing flexible jobs by a few hours substantially reduces emissions, most on solar-heavy grids [3]. Acun et al. study how renewable supply and storage choices shape data-centre carbon [4], and Sukprasert et al. analyse the limits of temporal and spatial shifting [5]. These works target hyperscale operators; we target an open, small-scale deployment.

B. Measuring and forecasting grid emissions

Siler-Evans et al. estimate marginal emission factors for U.S. regions and show they differ from averages [6]; Callaway et al. show the value of renewables and demand response depends on when and where they act [7]. We report *average* intensity and discuss the limitation in Section IV-G. CarbonCast forecasts multi-day grid carbon intensity with machine learning [8]; we deliberately use simpler, explainable methods and compare them to persistence. Commercial services (WattTime, Electricity Maps) sell carbon signals with mostly closed methods.

C. Stream processing

Kafka provides a durable log for high-volume feeds [9]. The Dataflow model [11] and Structured Streaming [10] formalise event-time windows and watermarks for late data; D-Streams [12] introduced micro-batch streaming on Spark. Kleppmann discusses the trade-offs of joining streaming and batch layers [13].

D. Streaming algorithms, hashing and privacy

Bloom filters give constant-space approximate set membership [15]; DGIM counts 1-bits in a sliding window in $O(\log^2 N)$ space [16]; reservoir sampling maintains a uniform sample of an unbounded stream [17]. Random-hyperplane LSH [18], [19] supports sub-linear similar-item retrieval. k-anonymity [20] and differential privacy [21] limit re-identification from released data.

E. Warehouse design

We follow dimensional modelling practice [14] with a Type-2 slowly changing dimension for emission factors.

III. System Design

A. Data sources

CAISO Today's Outlook provides generation by fuel and demand every five minutes; EIA-930 provides hourly generation by fuel for every balancing authority; NWS provides hourly temperature and sky-cover forecasts; CAISO OASIS provides day-ahead demand and wind/solar forecasts; EPA eGRID and CARB provide emission factors. We loaded 78 624 CAISO five-minute intervals (about 940 000 fuel-level rows) and about 426 000 EIA-930 hourly records covering 1 January–30 September 2026.

B. Carbon-intensity calculation

For interval t with generation $G_{f,t}$ (MW) and emission factor EF_f (kg CO_2/MWh),

$$CI_t = \frac{\sum_f \max(G_{f,t}, 0)\, EF_f}{\sum_f \max(G_{f,t}, 0)}. \tag{1}$$

Imports use CARB's unspecified-power default (0.428 t/MWh). CAISO reports negative battery output while charging; charging is load, so negative values are clipped to zero and removed from the denominator. Biogenic CO_2 is treated as zero **[TODO: state the eGRID year and factor table you used]**.

C. Ingestion and time handling

One Kafka topic per feed. CAISO publishes Pacific wall-clock time (with daylight saving) and EIA publishes UTC, so all timestamps are converted to UTC at ingestion; the repeated 01:00–02:00 hour on the autumn change is disambiguated by detecting time moving backwards. EIA returns at most 5000 rows per request, so the backfill pages each month by offset.

D. Streaming layer

Spark Structured Streaming reads Kafka, computes CI_t on five-minute tumbling event-time windows, and keeps the record with the greatest revision timestamp in each window under a 30-minute watermark, so a CAISO revision replaces the first publication. Micro-batches are written to Redis (latest interval only) and merged into Snowflake with a keyed `MERGE`, making the sink idempotent.

E. Streaming algorithms

Bloom filter. The producer re-reads the whole day file every five minutes. Two Bloom filters (interval keys; interval+content hash) classify each interval as new, revised or duplicate and suppress duplicates, using constant memory (m bits, k hashes; false-positive target 0.1%). *DGIM.* We count intervals above 250 kg/MWh in the last 288 (24 h) using DGIM buckets; the count is shown on the dashboard. *Reservoir sampling.* A 500-item uniform sample of intervals supports spot checks without storing the stream.

TABLE I
Accuracy of computed CO_2 against CAISO's published estimate.

Period	MAPE	Bias
[TODO: Sep 2026]	**[TODO: %]**	**[TODO: %]**

F. Warehouse

Snowflake holds raw, staging and mart layers built with dbt: `fact_grid_interval`, `fact_carbon_intensity`, `fact_ba_hourly`, `fact_forecast`, `fact_job_run`, and dimensions for fuel (SCD Type 2 via a dbt snapshot over the eGRID seed), time, balancing authority, forecast method and job. dbt tests cover uniqueness, non-null keys and a plausible-range check on intensity.

G. Forecasting

Persistence: tomorrow's hour equals today's. Net-load: the day-ahead demand forecast minus the wind and solar forecast gives net load; the forecast is the mean intensity of the $k = 15$ historical hours with the same Pacific hour and the nearest net load. LSH analog days: 24-hour net-load profiles are z-normalised and hashed with random hyperplanes (6 bits, 8 tables); the forecast is the mean intensity profile of the 7 nearest retrieved days.

H. Carbon-aware scheduling

An Airflow sensor reads the Redis ranking and releases a job when the current hour is among the three cleanest hours of the job's 12-hour window, or when the window is about to close (so work is never starved). Each run is logged with the intensity at run time, the intensity at the planned time, and kg CO_2 avoided.

I. Privacy

Because campus charger data was unavailable we simulate EV sessions, but the release path is real: direct identifiers are dropped, arrival time and energy are generalised, records in quasi-identifier groups smaller than $k = 5$ are suppressed, and hourly per-building totals receive Laplace noise ($\varepsilon = 1$, per-session clip 40 kWh).

IV. Evaluation

A. Setup

[TODO: machine specs, Docker resource limits, Spark local mode, date range of each experiment].

B. Latency

We sample the Redis value and measure the lag between the end of the interval and the time Redis is updated (an upper bound on publish-to-Redis latency). Target: under 10 min. Result: **[TODO: n, median, p95 from eval/latency_test.py]**.

C. Accuracy versus CAISO

Target: within 10% on average. **[TODO: state whether met; if not, explain the gap, e.g. factor choices, imports, units]**.

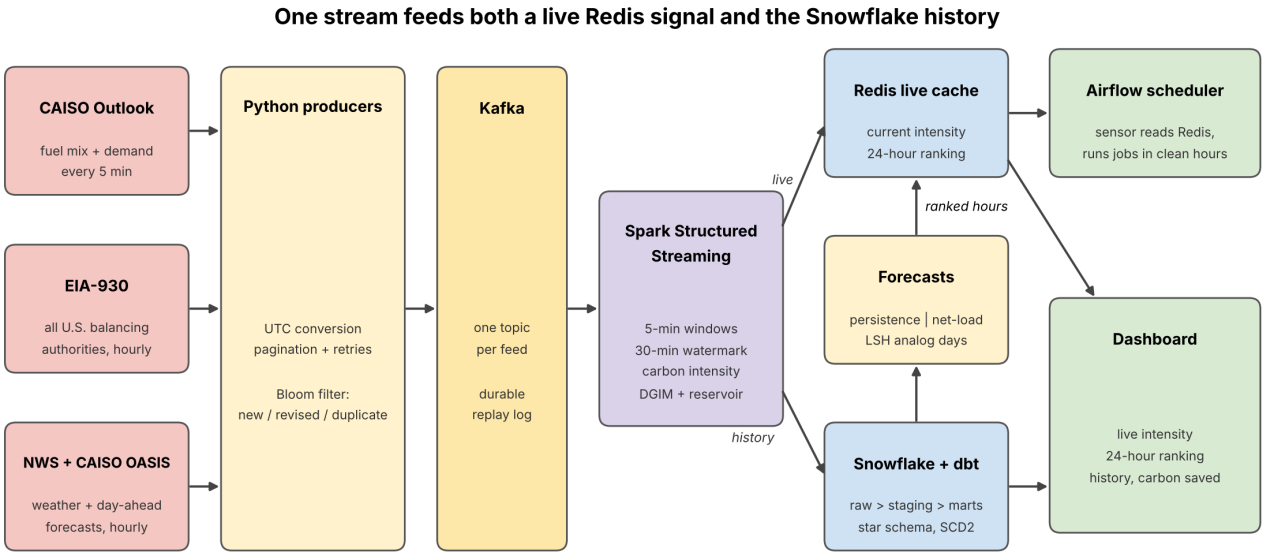


Fig. 1. Architecture. One stream feeds both the live Redis signal and the Snowflake history.

TABLE II
NEXT-DAY HOURLY INTENSITY FORECASTS (BACKTEST).

Method	Days	MAE (kg/MWh)	Top-4 overlap
Persistence	[TODO:]	[TODO:]	[TODO:]
Net-load	[TODO:]	[TODO:]	[TODO:]
LSH analog days	[TODO:]	[TODO:]	[TODO:]

[TODO: add figures/ev˙savings.png: from eval/savings_study.py on real history]

Fig. 2. Simulated EV charging: charge-on-arrival versus carbon-aware.

D. Late and revised data

[TODO: number of revised intervals observed (Bloom filter counters), share merged into the correct window = target 100%].

E. Forecast quality

State explicitly which net-load inputs were used (OASIS day-ahead forecasts, or realised net load plus noise in the backtest).

F. Carbon savings

[TODO: kg and % saved for simulated EV; kg avoided by logged Airflow runs; sensitivity to window length].

G. Limitations

Average intensity is not marginal intensity; shifting load can change which plants respond [6]. Campus meter data was simulated. CAISO feeds can be delayed or revised. **[TODO: add any limits you observed].**

V. Lessons Learned

[TODO: 3–5 honest lessons, e.g.: DST and time-zone alignment between CAISO and EIA; sign conventions for batteries; why single-partition topics keep watermarks correct during backfill; memory limits of running Kafka+Spark+Redis in Docker; idempotent merges for revised data; what you would do differently].

VI. Conclusion

[TODO: 3–4 sentences].

Team Contributions

S. Bhongle: Kafka producers, backfill, latency testing. S. K. Chintagunta: Spark streaming, carbon calculation, watermarks, Redis. V. K. Bethapudi: Snowflake, dbt models and tests, SCD Type 2. K. Routhu: forecasting, Airflow sensor, dashboard, savings analysis. **[TODO: adjust to what actually happened].**

Generative AI Disclosure

[TODO: list every tool used (ChatGPT for the status report, Claude for proposal topic/tools and for code scaffolding), what for, and attach the certificate, per course policy].

References

[1] California ISO, "Wind and solar curtailment report," May 30, 2025; "Key statistics," Mar. and May 2025.

[2] A. Radovanović *et al.*, "Carbon-aware computing for datacenters," *IEEE Trans. Power Syst.*, vol. 38, no. 2, 2023.

[3] P. Wiesner, I. Behnke, D. Scheinert, K. Gontarska, and L. Thamsen, "Let's wait awhile: How temporal workload shifting can reduce carbon emissions in the cloud," in *Proc. 22nd Int. Middleware Conf.*, 2021.

[4] B. Acun *et al.*, "A holistic approach for designing carbon aware datacenters (Carbon Explorer)," in *Proc. ASPLOS*, 2023.

[5] T. Sukprasert *et al.*, "On the limitations of carbon-aware temporal and spatial workload shifting in the cloud," in *Proc. EuroSys*, 2024.

[6] K. Siler-Evans, I. L. Azevedo, and M. G. Morgan, "Marginal emissions factors for the U.S. electricity system," *Environ. Sci. Technol.*, vol. 46, no. 9, pp. 4742–4748, 2012.

[7] D. S. Callaway, M. Fowlie, and G. McCormick, "Location, location, location: The variable value of renewable energy and demand-side efficiency resources," *J. Assoc. Environ. Resour. Econ.*, vol. 5, no. 1, pp. 39–75, 2018.

[8] D. Maji, P. Shenoy, and R. K. Sitaraman, "CarbonCast: Multi-day forecasting of grid carbon intensity," in *Proc. BuildSys*, 2022.

[9] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proc. NetDB Workshop*, 2011.

[10] M. Armbrust *et al.*, "Structured Streaming: A declarative API for real-time applications in Apache Spark," in *Proc. ACM SIGMOD*, 2018.

[11] T. Akidau *et al.*, "The Dataflow Model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing," *Proc. VLDB Endow.*, vol. 8, no. 12, 2015.

[12] M. Zaharia *et al.*, "Discretized streams: Fault-tolerant streaming computation at scale," in *Proc. ACM SOSP*, 2013.

[13] M. Kleppmann, *Designing Data-Intensive Applications*. O'Reilly Media, 2017.

[14] R. Kimball and M. Ross, *The Data Warehouse Toolkit*, 3rd ed. Wiley, 2013.

[15] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Commun. ACM*, vol. 13, no. 7, pp. 422–426, 1970.

[16] M. Datar, A. Gionis, P. Indyk, and R. Motwani, "Maintaining stream statistics over sliding windows," *SIAM J. Comput.*, vol. 31, no. 6, pp. 1794–1813, 2002.

[17] J. S. Vitter, "Random sampling with a reservoir," *ACM Trans. Math. Softw.*, vol. 11, no. 1, pp. 37–57, 1985.

[18] P. Indyk and R. Motwani, "Approximate nearest neighbors: Towards removing the curse of dimensionality," in *Proc. STOC*, 1998.

[19] M. S. Charikar, "Similarity estimation techniques from rounding algorithms," in *Proc. STOC*, 2002.

[20] L. Sweeney, "k-anonymity: A model for protecting privacy," *Int. J. Uncertainty, Fuzziness Knowl.-Based Syst.*, vol. 10, no. 5, pp. 557–570, 2002.

[21] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Found. Trends Theor. Comput. Sci.*, vol. 9, no. 3–4, pp. 211–407, 2014.